Introduction to Machine Learning

Problem Sheet 6 - Python projects
4th year Statistics and Data Science

Ayoub Asri

15 March 2026

Problem Sheet VI

Project 01:

The goal of this practice is to apply t-SNE and UMAP from scratch.

Use the file provided 'Project 01 skeleton code.ipynb" as a skeleton code to reproduce all necessary codes.

t-SNE

Q1. Load the data provided in the file "tsne.txt". Inspect the data ? precise the dimension of the data. Perform a log transformation to all the variables (be aware of the existence of 0s)

Q2. create an object containing the distance squared between all observations

Q3. define the function to calculate the probability in high dimension $p_{j|i}$ and perplexity defined as a Shannon cross entropy.

$$\text{Perplexity} = 2^{-\sum_j p_{j|i} \log p_{j|i}}$$

Q4. complete the function to perform binary search for the value of σ to solve the equation

$$\text{Perplexity}(\sigma) \approx \text{Perplexity}_{fixed}$$

Q5. apply the binary search algorithm for the case of a perplexity value equal to 30.

You have to observe that the σ value will be calculated for each row. The final value of σ will be the mean of all values1

Q6. calculate the p_{ij}

Q7. create the function to calculate the probability on the target space $q_{j|i}$

Q8. define the two functions to calculate the kL divergence and the KL divergence gradient.

Q9. apply the gradient descent algorithm with a learning rate = 0.6 and number of iterations = 200.

modify this function to include the projection in the lower dimensional space for each iteration (no need to show the plot of each iteration you can just save the plot)

Q10. plot the low dimensionsial space for the last iteration

Q11. (optional) you can combine all the plots for the iterations into one GIF animation to follow the steps of the algorithm.

Q12. compare the results of the defined algorithm with the implementation of tSNE in scikit-learn. Does it reproduce the same solution ? what hyperparameters should we modify to get the same solution ?

UMAP

before applying the UMAP we will discuss the use of the family of distribution used in UMAP

$$Q(y) = (1 + ay^{2b})^{-1}$$

Q1. plot the function for different values of a and b in the same plot (you have to use the default values : $a = 1.93, b = 0.79$)

Q2. to illustrate the use of `min_dist` hyperparameter we will illustrate this transformation in UMAP

$$(1 + a(y_i - y_j)^{2b})^{-1} \approx \begin{cases} 1 & y_i - y_j \leq mindist \\ e^{-(y_i - y_j) - mindist} & y_i - y_j > mindist \end{cases}$$

we will the function **optimise.curve_fit** of scipy to compare the two distributions. Use the code provided and try different values of min_dist and study the effect of the hyperparameters a and b

Q3. apply the spectral embedding algorithm on the data (use SpectralEmbedding from sklearn.manifold)

Now we will apply the UMAP

Q4. reproduce the functions to find the value σ depending on a predefined number of neighbors

apply the procedure with number of neighbors = 15

Q5. determine the values of a and b for a minimum distance equal a 0.25

Q6. define the probabilities in low space, calcualte the Cross Entropy and its gradient

Q7. apply the gradient descent algorithm with a learing rate = 1 and maximum iteration = 200

Q8. plot the variation of the Cross Entropy for each itraraion

Q9. plot the final iteration projection

Q10. compare the results to the implementation of UMAP in the umap library

Q11. try different variations of the number of neighbors and min distance parameters

Application on MNIST

apply both t-SNE and UMAP with different variations of hyperparameters on the MNIST dataset.

Project 02:

The goal of this practice is the implementation of **clustering** algorithms. Clustering is an **unsupervised** problem. Given a training set $x_1, \ldots, x_n$ of points in $\mathbb{R}^d$, the aim is to group the points into clusters (ideally homogeneous groups), or **clusters**.

Note that unlike in a supervised problem, there are no labels associated in this context. . ### K-Means

In this section, we will focus on algorithms where the number of clusters (denoted as K) is predefined by the user.

The K-Means algorithm partitions the points into K disjoint groups $\{C_1, \ldots, C_K\}$ by minimizing the **intra-cluster variance**. The criterion minimized, G, is called the inertia:

$$G_X(C_1, \ldots, C_K) = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|_2^2$$

where $\mu_k \in \mathbb{R}^d$ are the **centroids** of the K classes ($|C_k|$ being the cardinality of each class):

$$\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} x_i, \ \forall k \in [K],$$

and where the i-th observation is assigned to the class of the nearest centroid, i.e., $i \in C_j$ if and only if:

$$j = \arg\min_{k \in [K]} \|x_i - \mu_k\|_2 = \arg\min_{k \in [K]} \|x_i - \mu_k\|_2^2.$$

Note: In case of ties, we break them randomly if necessary.

The learning of the clusters is done by **alternating iteratively** between the following two steps:

K-Means Algorithm: Detailed Steps

I) **Assignment Step**: Given the centroids $(\mu_k)_{k=1,\ldots,K}$, determine the labels for each point.

II) **Centroid Update Step**: Given the labels, update the centroids by determining the class centers.

The algorithm is stopped when the inertia no longer decreases significantly (note that even without a stopping criterion, the K-Means algorithm will always terminate after a finite number of steps).

Since inertia is a non-convex criterion, the solution depends on the initialization, as is often the case for such problems. As a result, the algorithm is often run multiple times with different initializations, and the solution with the lowest inertia is retained.

Questions:

Q1. Based on the code skeleton in the "Project 02 skeleton code.ipynb" file, implement the K-Means algorithm. The file ends with an example code that will help you with validation. It uses the make_blobs function to simulate a dataset.

Q2. Does the inertia decrease during iterations? You will display the decrease in inertia as a function of the iterations.

Q3. Effect of Initialization: By varying the initialization of the random number generator, observe that the solution found is not always the same. You can increase the number of simulated clusters to further highlight this phenomenon.

Q4. Compare your implementation with that of scikit-learn in terms of results and computation time. Here is an example of using scikit-learn:

```
from sklearn import cluster
kmeans = cluster.KMeans(n_clusters=3, n_init=10)
kmeans.fit(X)
labels = kmeans.labels_
```

Optimal Calculation of the Number of Clusters

In the previous section, we assumed that the number of clusters K was set by the user. However, it may be practical to determine the optimal number of clusters. We propose a method to calculate it.

We are still working with a dataset X with n points. Let $X_t, t \in [T]$, be T independent draws of n random points drawn according to a uniform distribution within the bounding box of X. Based on the inertias G_X and G_{X_t} of the K-means applied to X and X_t, we define the difference:

$$\delta(t) = \text{Gap}(t) - (\text{Gap}(t+1) - \sigma(t+1))$$

where $\text{Gap}(t) \in \mathbb{R}$ corresponds to the difference between the expectation of the logarithm of the inertias of $G_{X_t}$and the logarithm of the inertia G_X:

$$\text{Gap}(t) = E[\log(G_{X_t})] - \log(G_X)$$

and where $\sigma(t)$ is defined by:

$$\sigma(t) = \sqrt{\frac{T+1}{T}\text{std}(\log(X_t))}$$

with std representing the standard deviation.

Questions

Q1. Complete the function `compute_log_inertia_rand` so that it estimates the mean and standard deviation of $\log(G_{X_t})$.

Q2. Complete the function `compute_gap` which calculates, for a fixed dataset X, the values of $\text{Gap}(t)$ and $\delta(t)$ for different values of t.

Q3. Analyze the values of$\text{Gap}(t)$ and $\delta(t)$ for different values of k and propose a method to automatically determine the number K for K-means. You will use the `sklearn.datasets.make_blobs` function to generate a dataset of K clusters and the `plot_gap` function to visualize your different data.

Q4. Complete the function `optimal_n_clusters_search` with the method you proposed and verify its functionality on different datasets.

Application to Image Compression

Clustering algorithms can also be used for vector quantization. An image is represented by a tensor of dimension 3. The image has $n_1 \times n_2$ pixels, and for each pixel $ (x_1, x_2)$, the associated value is a triplet $(I_R, I_G, I_B) \in \mathbb{R}^3$ that indicates the intensity of the three colors with the following encoding:

- I_R which codes for the intensity of red
- I_G which codes for the intensity of green
- I_B which codes for the intensity of blue

The color intensity for each pixel is encoded by a byte: an integer between 0 and 255 ($ = 2^8 - 1$).

Note: Sometimes RGBA is also used, which includes an additional term I_A that codes the transparency level (often referred to as alpha, which is encoded here as a float on $[0, 1]$).

The idea of vector quantization is to code each pixel using only K colors, with K being much smaller than the 256^3 possible values. The image can thus be encoded more compactly.

Questions

We propose to compare the results obtained with different compression techniques:

Q1. Apply the K-Means algorithm to reduce the number of color levels in the image to $K = 8$ possible values. Visualize the result as an image. From what value of K do we no longer see a difference compared to the original image?

Q2. (optional) Apply the spectral clustering algorithm to reduce the number of color levels in the image to 8 possible values. Note that, for performance and memory usage reasons, it is essential to use the spectral clustering version that employs sparse matrices. Visualize and compare the results obtained with K-Means.

Hint: To read the provided image 'china.jpg', you can use the following command:

```
import matplotlib.pyplot as plt
img = plt.imread('china.jpg')
```

You will also need to perform a reshape of the image in order to work with a 2-dimensional array of size $n = n_1 \times n_2 \times 3$ (in fact, there are three colors for RGB encoding, and we will ignore the alpha level).

Project 03

The aim of this project is to implement the Expectation-Maximization (EM) Algorithm and apply it to various cases, including:

- Gaussian Mixture Models (GMMs)
- K-means
- Probabilistic PCA (PPCA)

For this project, please refer to the attached Jupyter Notebook "Project 03 skeleton code.ipynb", which contains the provided skeleton code.

Part 1: Gaussian Mixture Models (GMM)

In this section, we will focus on fitting GMMs using the EM algorithm.

Before diving into the algorithm, let's create a function to plot the Probability Density Function (PDF) and the histogram of any data vector.

Q1. Complete the `plot_pdf` function to generate a Gaussian sample and plot its histogram along with its density plot.

Q2. Use this function to generate three different Gaussian distributions, each with 100 observations, using the following parameters:

For the first distribution:

$$\mu_1 = -5, \sigma_1 = 1.2$$

For the second distribution:

$$\mu_2 = 5, \sigma_2 = 1.8$$

For the third distribution:

$$\mu_3 = 0, \sigma_3 = 1.6$$

Plot all three distributions on the same figure. These values will be used to test the model.

Q3. Create a variable `X` that contains all three normal variables generated in Q2.

Now, let's focus on building the EM algorithm.

Q4. Complete the function `random_init` to initialize the parameters μ_i, σ_i and p_i.. This function should also plot the Gaussian distributions generated by this random initialization.

Hint: The means and variances are initialized with random values, while the p_i should be initialized to $1/n$.

Q5. Complete the functions `step_expectation` and `step_maximization`, which correspond to the two steps of the EM algorithm.

Q6. Utilize all the defined functions, along with the data generated in Q2 and the predefined function `plot_intermediate_steps`, to complete the function `train_gmm`. This function should execute the algorithm and plot the various results.

Q7. Comment on your results. Does the model always converge? What factors contribute to non-convergence?

Part 2: K-means

In this part, we will apply the same logic to reproduce the K-means clustering example using the EM algorithm.

Q1. Use the provided code in the function `generate_kmeans_data` to generate a toy dataset.

Q2. Complete all the remaining functions and apply K-means to the generated dataset.

Q3. Make the necessary modifications to the functions and apply K-means to the provided China image.

Part 3: Probabilistic PCA (PPCA)

Q1. Apply the PPCA to the generated data provided by the function `generate_ppca_data`.

Q2. (Optional) Apply the PPCA to the MNIST dataset. Use the provided code to read the data and implement PPCA. Please include your comments on the results.